

回顾

插入排序多种实现算法：

- | | |
|------------|---------------------|
| 1) 直接插入排序 | $O(n^2)$ |
| 2) 折半插入排序 | $O(n^2)$ |
| 3) 2-路插入排序 | $O(n^2)$ |
| 4) 表插入排序 | $O(n^2)$ |
| 5) 希尔排序 | 与增量相关 $O(n^{1.25})$ |

第10章 内部排序

10.1 概述

10.2 插入排序

10.3 交换排序

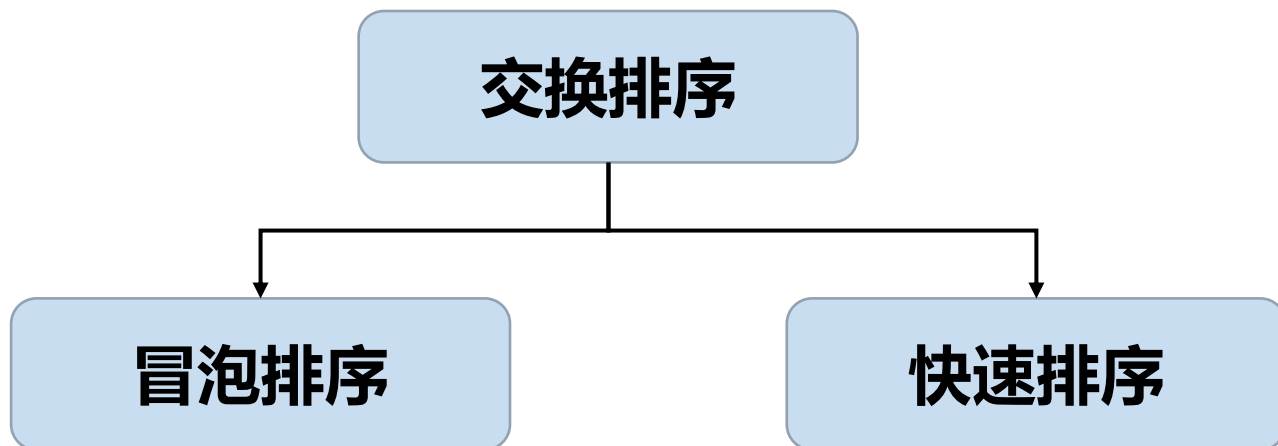
10.4 选择排序

10.5 归并排序

10.6 基数排序

10.3 交换排序

思想：两两比较待排序记录的关键码，如果发生逆序（即排列顺序与排序后的次序正好相反），则交换之，直到所有记录都排好序为止。



10.3.1 冒泡排序

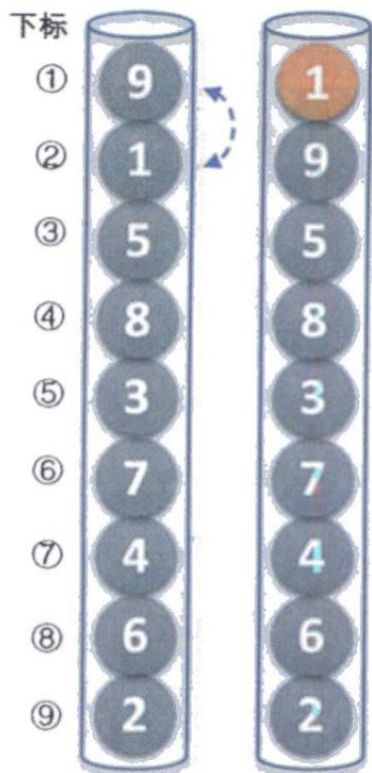
基本思路：每趟不断将相邻两记录两两比较，并按“前小后大”（或“前大后小”）规则交换。

优点：每趟结束时，不仅能挤出一个最大值到最后面位置，还能同时部分理顺其他元素；一旦下趟没有交换发生，还可以提前结束排序。

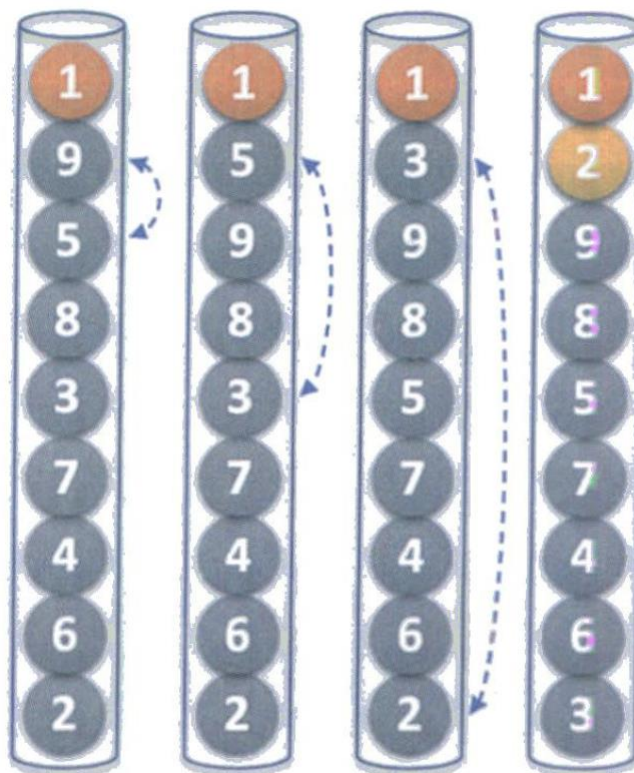
前提：顺序存储结构

10.3.1 冒泡排序

关键字序列 $T=(9, 1, 5, 8, 3, 7, 4, 6, 2)$



当 $i=1$ 时, 9与1交换后, 1与其余关键字比较, 都是最小, 因此1即最小值放置在首位



当 $i=2$ 时, 9与5, 5与3, 3与2交换, 最终将2放置在第二位

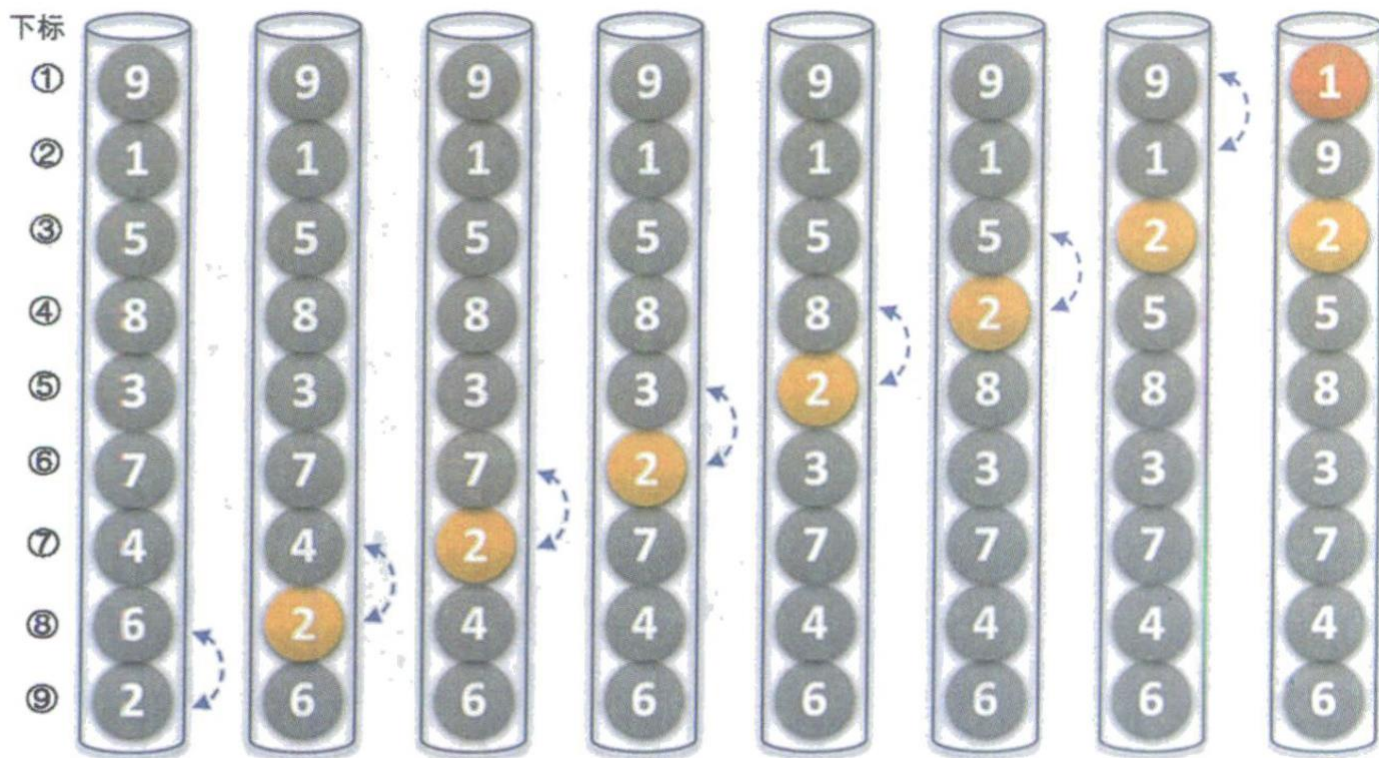
10.3.1 冒泡排序

冒泡排序初级版

```
void BubbleSort0(SgList *L)
{
    int i, j;
    for(i=1; i<L->length; i++)
    {
        for(j=i+1; j<=L->length; j++)
        {
            if(L->r[i] > L->r[j])
            {
                swap(L, i, j); /* 交换L->r[i]与L->r[j]的值 */
            }
        }
    }
}
```

10.3.1 冒泡排序

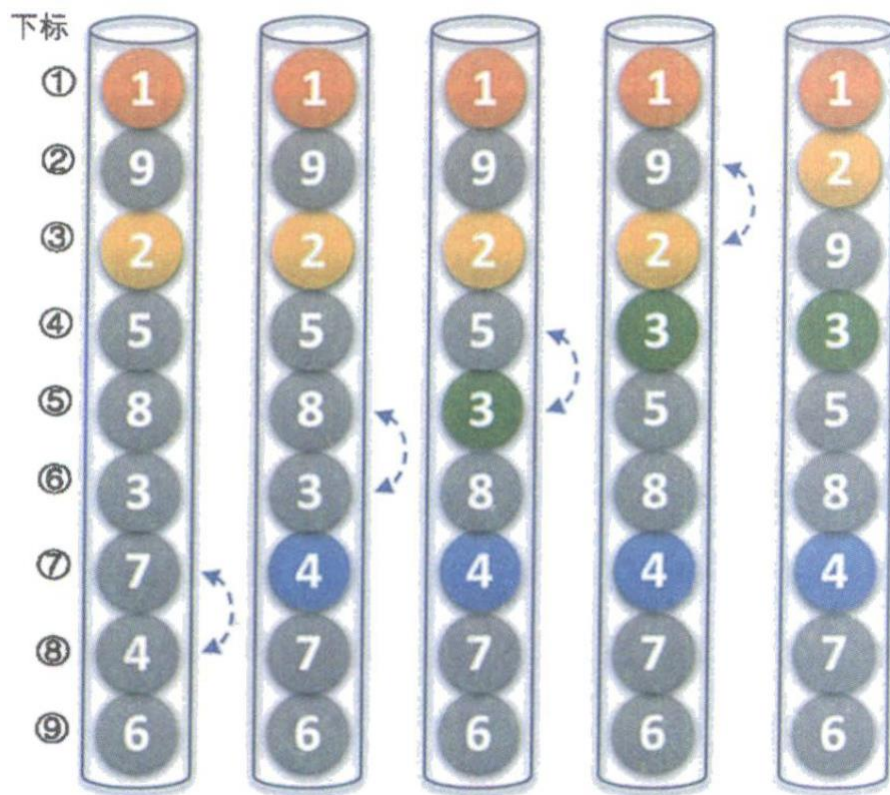
关键字序列 $T=(9, 1, 5, 8, 3, 7, 4, 6, 2)$



当 $i=1$ 时，将最小值 1 冒泡到顶端

10.3.1 冒泡排序

关键字序列 $T=(9, 1, 5, 8, 3, 7, 4, 6, 2)$



当 $i=2$ 时，将次小值2冒泡到第2位置

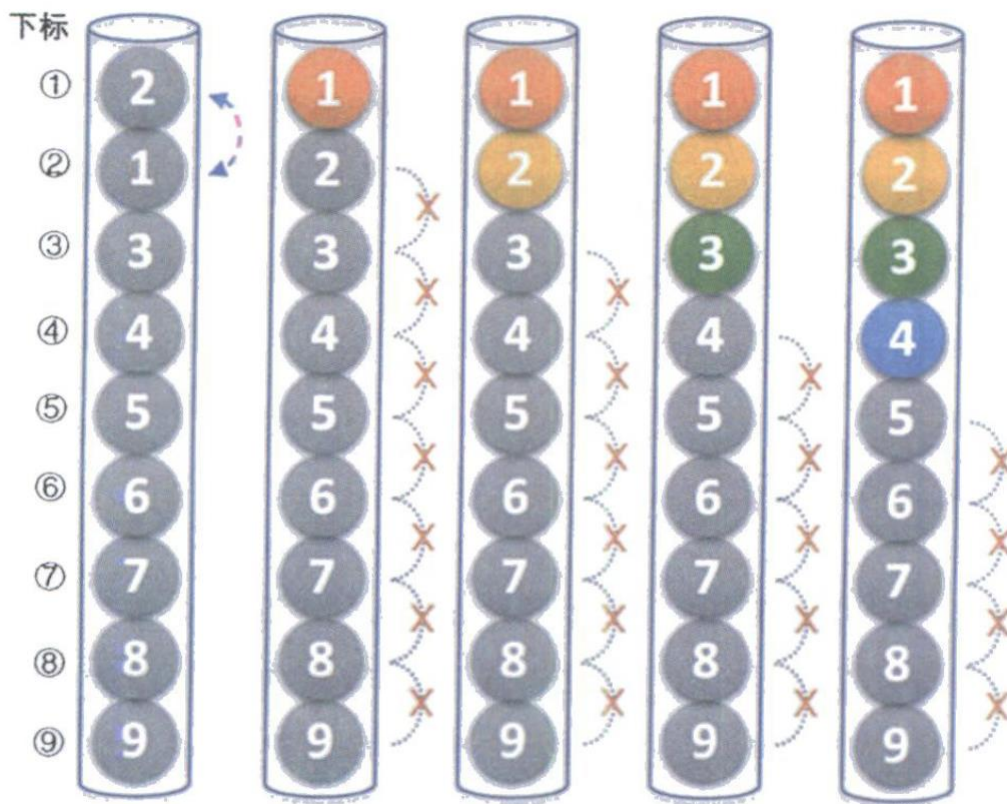
10.3.1 冒泡排序

冒泡排序正式版

```
void BubbleSort(SgList *L) /*对顺序表L作冒泡排序 */
{
    int i, j;
    for(i=1; i<L->length; i++)
    {
        for(j=L->length-1; j>=i; j--) /* 注意j是从后往前循环 */
        {
            if(L->r[j] > L->r[j+1]) /* 若前者大于后者 (注意这里与上一算法的差异) */
            {
                swap(L, j, j+1); /* 交换L->r[j]与L->r[j+1]的值 */
            }
        }
    }
}
```

10.3.1 冒泡排序

关键字序列 $T=(2, 1, 3, 4, 5, 6, 7, 8, 9)$



当 $i=2$ 时，由于没有任何数据交换，就说明此序列已经有序，之后的循环判断都是多余的

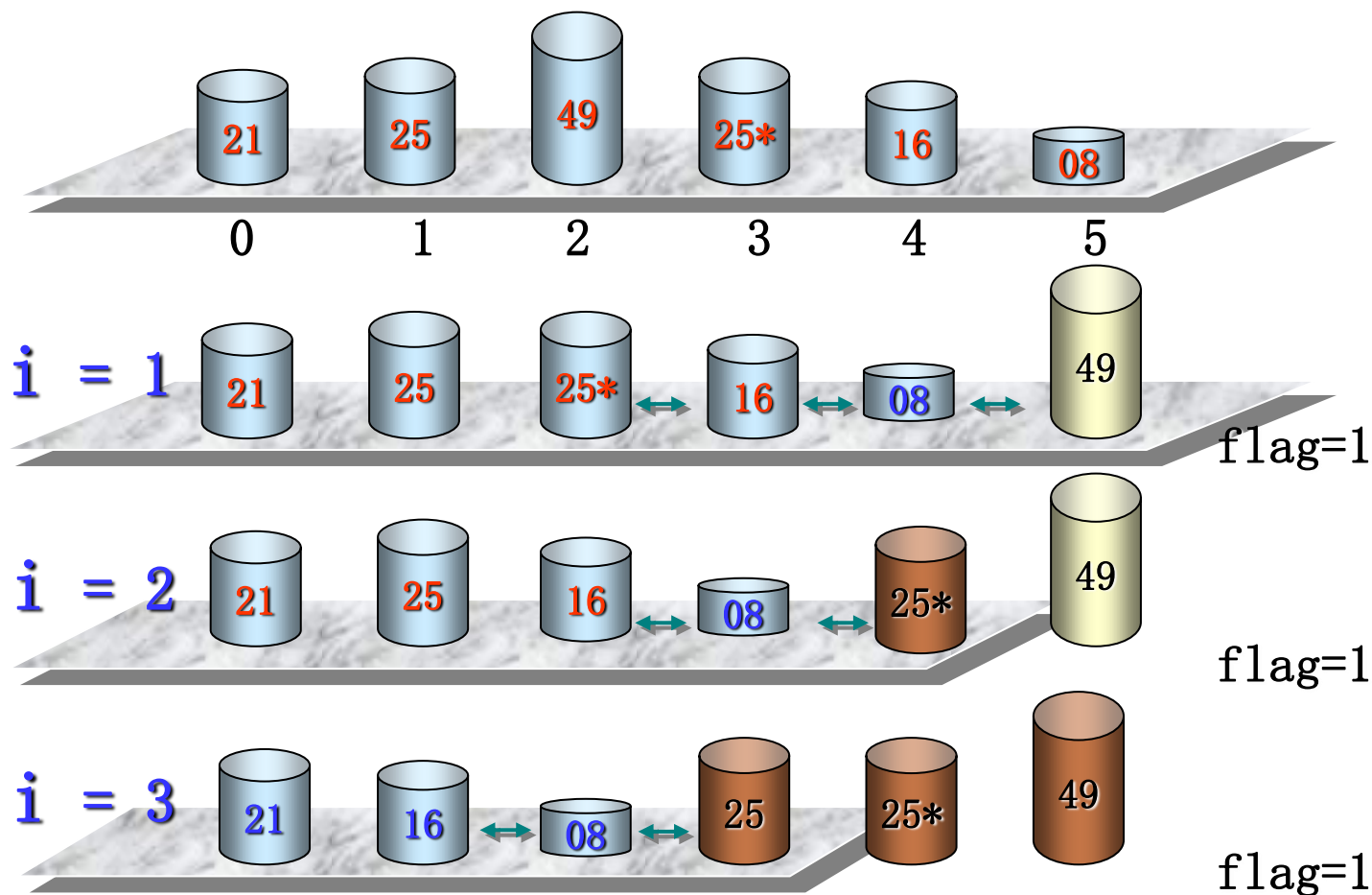
10.3.1 冒泡排序

设置flag用来标志后面的数字是否已经完成排序

```
void BubbleSort2(SqList *L) { /* 对顺序表L作改进冒泡算法 */
    int i, j;
    Status flag=TRUE; /* flag用来作为标记 */
    for(i=1;i<L->length && flag;i++) { /* flag为TRUE则有数据交换，否则退出循环*/
        flag=FALSE; /* 初始为FALSE */
        for(j=L->length-1;j>=i;j--) {
            if(L->r[j] > L->r[j+1]) {
                swap(L,j,j+1); /* 交换L->r[i]与L->r[j+1]的值 */
                flag=TRUE; /* 如果有数据交换，则flag为TRUE */
            }
        }
    }
}
```

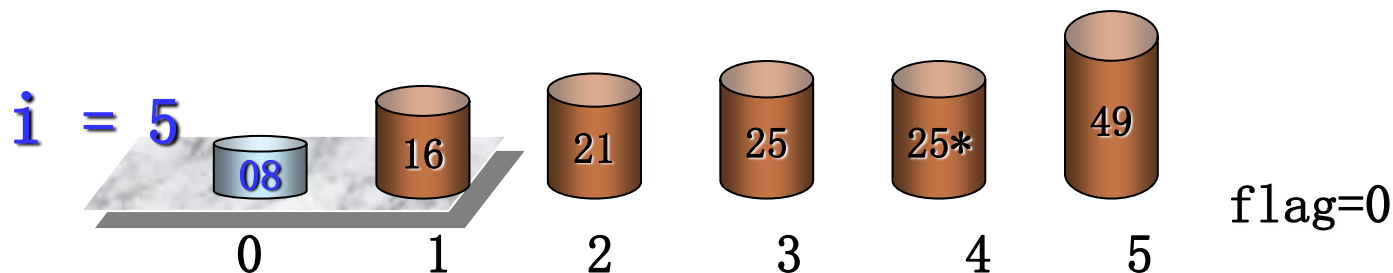
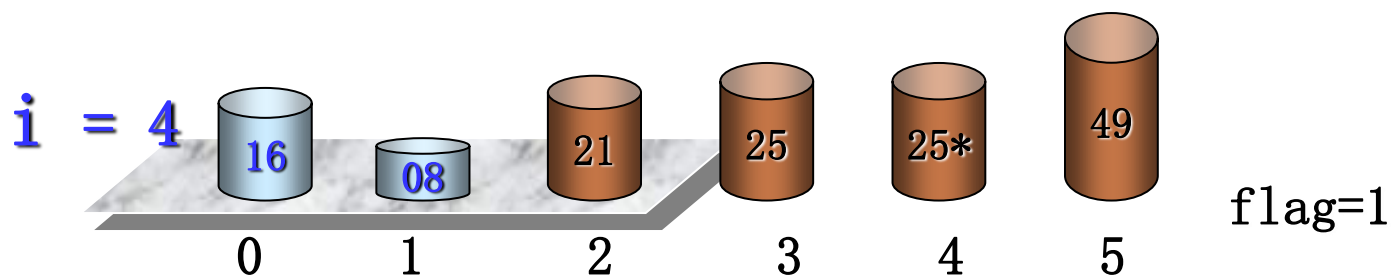
10.3.1 冒泡排序

关键字序列 $T=(21, 25, 49, 25^*, 16, 08)$ ，请写出冒泡排序的具体实现过程。



10.3.1 冒泡排序

关键字序列 $T=(21, 25, 49, 25^*, 16, 08)$ ，请写出冒泡排序的具体实现过程。



10.3.1 冒泡排序

算法2:

```
void BubbleSort2(SqList &L){
    int i, j;
    Status flag=TRUE
    for(i=L.length; i>1 && flag; --i){
        flag = FALSE;
        for (j=1; j<i; ++j){
            if (LT(L.r[j+1].key, L.r[j].key)){
                ElemType temp=L.r[j];
                L.r[j]=L.r[j+1];
                L.r[j+1]=temp;
                flag = TRUE;
            }
        }
    }
}
```

10.3.1 冒泡排序

- 至少比较1次，至多比较 $n-1$ 次
- 一次确定位置
- 可实现部分排序
- 稳定排序

10.3.1 冒泡排序

冒泡排序的算法分析

- 最好情况：初始排列已经有序，只执行一趟起泡，做 $n-1$ 次关键码比较，不移动对象。
- 最坏情形：初始排列逆序，算法要执行 $n-1$ 趟起泡，第 i 趟 ($1 \leq i < n$) 做了 $n-i$ 次关键码比较，执行了 $n-i$ 次对象交换。

此时的比较总次数 KCN :

$$KCN = \sum_{i=1}^{n-1} (n-i) = \frac{1}{2}n(n-1)$$

10.3.1 冒泡排序

时间效率: $O(n^2)$ — 因为要考虑最坏情况

空间效率: $O(1)$ — 只在交换时用到一个缓冲单元

稳定性: 稳定 — 25和25*在排序前后的次序未改变

10.3.1 冒泡排序

冒泡排序的优点：每一趟整理元素时，不仅可以完全确定一个元素的位置（挤出一个泡到表尾），还可以对前面的元素作一些整理，所以比一般的排序要快。

有没有比冒泡排序更快的算法？

有！

快速排序法——全球公认！

因为它每趟都能准确定位不止1个元素！

10.3.2 快速排序

D&C分治思想：

把一个较大规模的问题拆分为若干小规模且相似的问题。再对小规模问题进行求解，最终合并所有小问题的解，从而形成原来大规模问题的解。

循环递归的过程，每一层递归有三个具体步骤：

- ❑ **分解：**将原问题分解为若干个规模较小，相对独立，与原问题形式相同的子问题。
- ❑ **解决：**若子问题规模较小且易于解决时，则直接解。否则，递归地解决各子问题。
- ❑ **合并：**将各子问题的解合并为原问题的解

10.3.2 快速排序

基本思想：

从待排序列中任取一个元素 (例如取第一个) 作为轴心(pivot)，所有比它小的元素一律前放，所有比它大的元素一律后放，形成左右两个子表；

然后再对各子表重新选择轴心元素并依此规则调整，直到每个子表的元素只剩一个。此时便为有序序列了。

10.3.2 快速排序

需解决的关键问题？

- (1) 如何选择轴值（轴心的值）？
- (2) 如何实现分割(称一次划分)？
- (3) 如何处理分割得到的两个待排序子序列？
- (4) 如何判别快速排序的结束？

10.3.2 快速排序

关键问题(1):如何选择轴值?

选择轴值的方法:

- ① 使用第一个记录的关键码;
- ② 选取序列中间记录的关键码;
- ③ 比较序列中第一个记录、最后一个记录和中间记录的关键码, 取关键码居中的作为轴值并调换到第一个记录的位置;
- ④ 随机选取基准值。

选取不同轴值的后果

决定两个子序列的长度, 子序列的长度最好相等

10.3.2 快速排序

关键问题(2):如何实现一次划分?

解决方法:

设待划分的序列是 $r[s] \sim r[t]$, 设参数 i, j 分别指向子序列左、右两端的下标 s 和 t , 令 $r[s]$ 为轴值。

(1) j 从后向前扫描, 直到 $r[j] < r[i]$, 将 $r[i]$ 移动到 $r[j]$ 的位置, 使关键码小(同轴值相比)的记录移动到前面去;

(2) i 从前向后扫描, 直到 $r[i] > r[j]$, 将 $r[i]$ 移动到 $r[j]$ 的位置, 使关键码大(同轴值比较)的记录移动到后面去;

(3)重复上述过程, 直到 $i=j$ 。

10.3.2 快速排序

关键问题(2):如何实现一次划分?

一次划分描述:

```
int partition(int a[],int low,int high ){
    i= low; j= high; pivotkey=a[i];
    while(i<j){
        while(i<j && pivotkey <= a[j].key) --j;
        if(i<j){ //由于pivotkey>a[j].key退出前面的循环, 则交换, 并调整i
            a[i]<-->a[j]; i++;
        }
        while(i<j && a[i].key <= pivotkey) i++;
        if(i<j){ //由于a[i].key>pivotkey退出前面的循环, 则交换, 并调整i
            a[i]<-->a[j]; j--;
        }
    }
    return i; //i==j时, 返回轴值
}
```

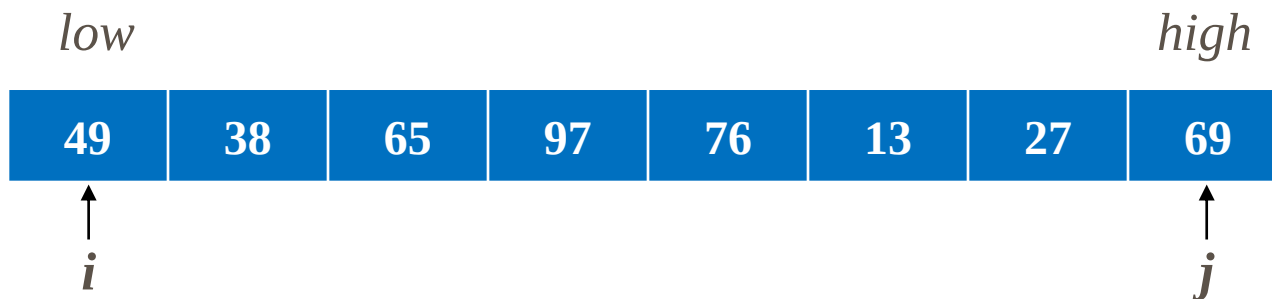
10.3.2 快速排序

初始化:把第一个元素看成是中轴元素

```
i = low;
```

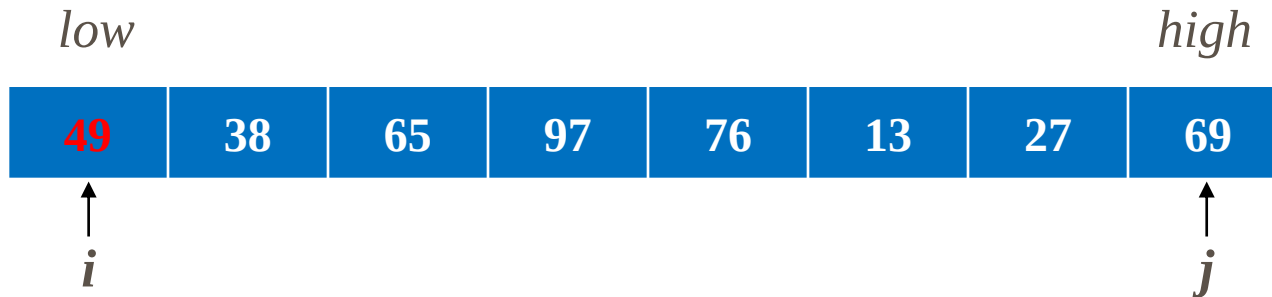
```
j = high;
```

```
pivotkey = a[i];
```



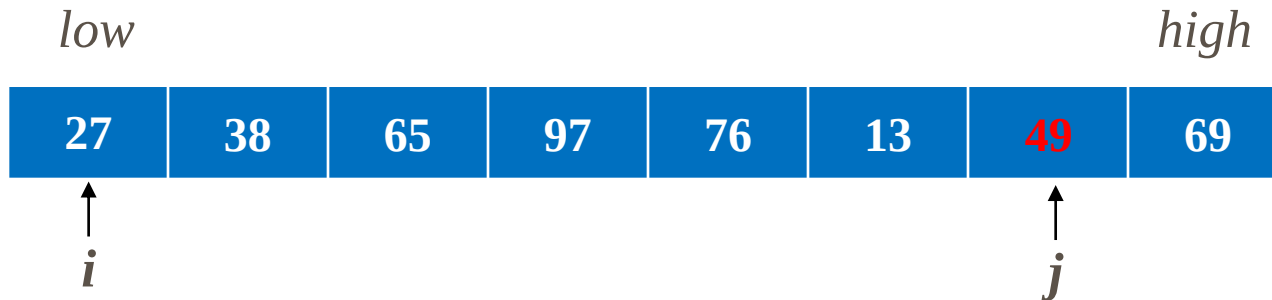
10.3.2 快速排序

```
while(i<j){  
    while(i<j&& pivotkey <= a[j].key) --j;  
    if(i<j) //由于pivotkey>a[j].key退出前面的循环, 则交换, 并调整i  
    { a[i]<-->a[j]; i++; }  
    while(i<j && a[i].key< = pivotkey) i++;  
    if(i<j) //由于a[i].key>pivotkey退出前面的循环, 则交换, 并调整i  
    { a[i]<-->a[j]; j--; }  
}
```



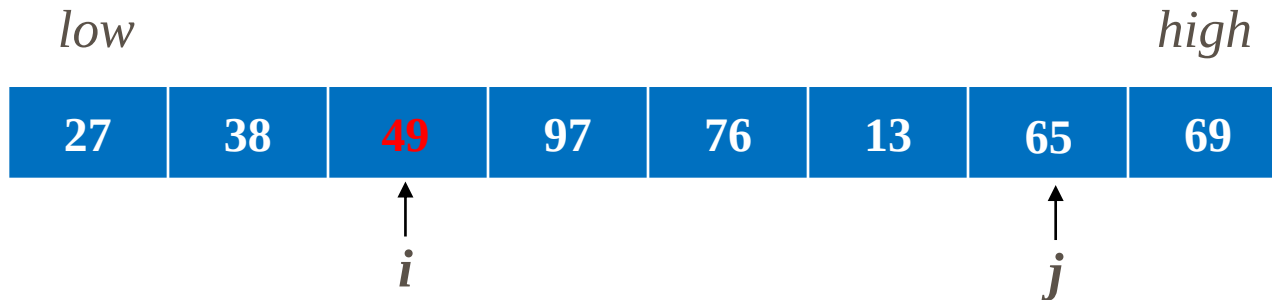
10.3.2 快速排序

```
while(i<j){  
    while(i<j&& pivotkey <= a[j].key) --j;  
    if(i<j) //由于pivotkey>a[j].key退出前面的循环, 则交换, 并调整i  
    { a[i]<-->a[j]; i++; }  
    while(i<j && a[i].key <= pivotkey) i++;  
    if(i<j) //由于a[i].key>pivotkey退出前面的循环, 则交换, 并调整i  
    { a[i]<-->a[j]; j--; }  
}
```



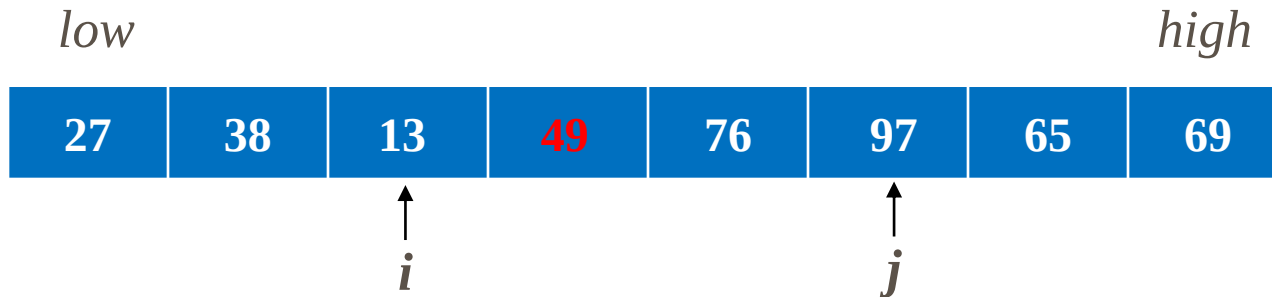
10.3.2 快速排序

```
while(i<j){  
    while(i<j&& pivotkey<= a[j].key) --j;  
    if(i<j) //由于pivotkey>a[j].key退出前面的循环, 则交换, 并调整i  
    { a[i]<-->a[j]; i++; }  
    while(i<j && a[i].key<=pivotkey) i++;  
    if(i<j) //由于a[i].key>pivotkey退出前面的循环, 则交换, 并调整i  
    { a[i]<-->a[j]; j--; }  
}
```



10.3.2 快速排序

```
while(i<j){  
    while(i<j&& pivotkey<= a[j].key) --j;  
    if(i<j) //由于pivotkey>a[j].key退出前面的循环, 则交换, 并调整i  
    { a[i]<-->a[j]; i++; }  
    while(i<j && a[i].key<=pivotkey) i++;  
    if(i<j) //由于a[i].key>pivotkey退出前面的循环, 则交换, 并调整i  
    { a[i]<-->a[j]; j--; }  
}
```



10.3.2 快速排序

关键问题(2):如何实现一次划分?

一次划分改进:

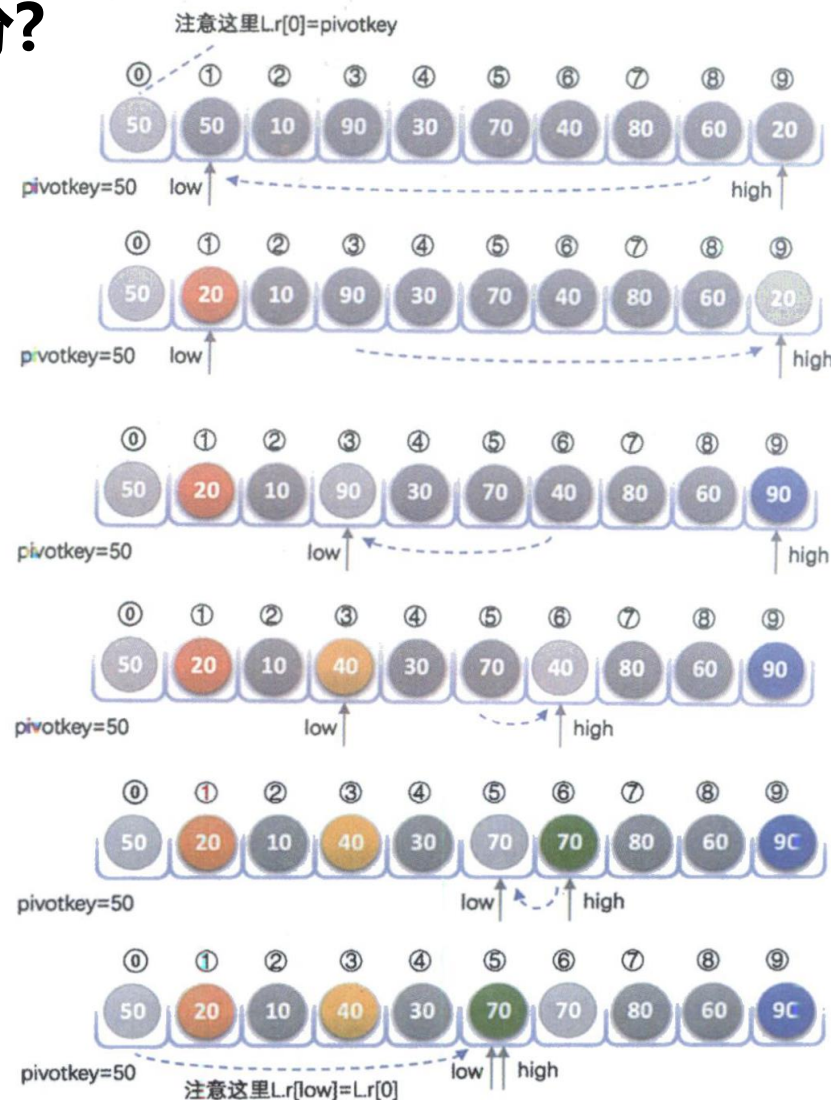
```
int partition(int a[],int low,int high ){
    i= low; j= high; pivotkey=a[i];
    a[0] = a[i];
    while(i<j){
        while(i<j&& pivotkey<= a[j].key) --j;
        if(i<j){ //由于pivotkey>a[j].key退出前面的循环, 则交换, 并调整i
            a[i] = a[j]; i++;
        }
        while(i<j && a[i].key<=pivotkey) i++;
        if(i<j){ //由于a[i].key>pivotkey退出前面的循环, 则交换, 并调整i
            a[j] = a[i]; j--;
        }
    }
    a[i] = a[0];
    return i; //i==j时, 返回轴值
}
```

10.3.2 快速排序

关键问题(2):如何实现一次划分?

一次划分改进:

```
int partition(int a[],int low,int high ){
    i= low; i= high; pivotkey=a[i];
    a[0] = a[i];
    while(i<j){
        while(i<j&& pivotkey<= a[j].key) --j;
        if(i<j){
            a[i] = a[j]; i++;
        }
        while(i<j && a[i].key<=pivotkey) i++;
        if(i<j){
            a[j] = a[i]; j--;
        }
    }
    a[i] = a[0];
    return i; //i==j时, 返回轴值
}
```



10.3.2 快速排序

关键问题(3):获取轴值划分后, 如何处理分割得到的两个待排序子序列?

解决方法:

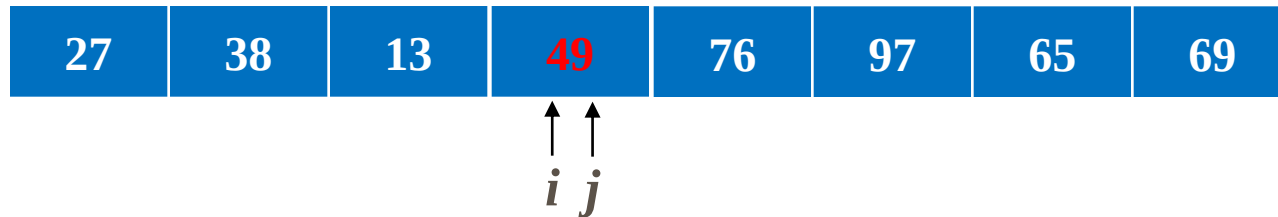
对分割得到的两个子序列递归地执行快速排序

算法描述:

```
void quickSort(int r[], int low, int high )
{
    pos = partition(r, low, high ); //一次划分
    quickSort(r, low, pos-1); //对前一个子序列进行快速排序
    quickSort(r, pos+1,high ); //对后一个子序列进行快速排序
}
```

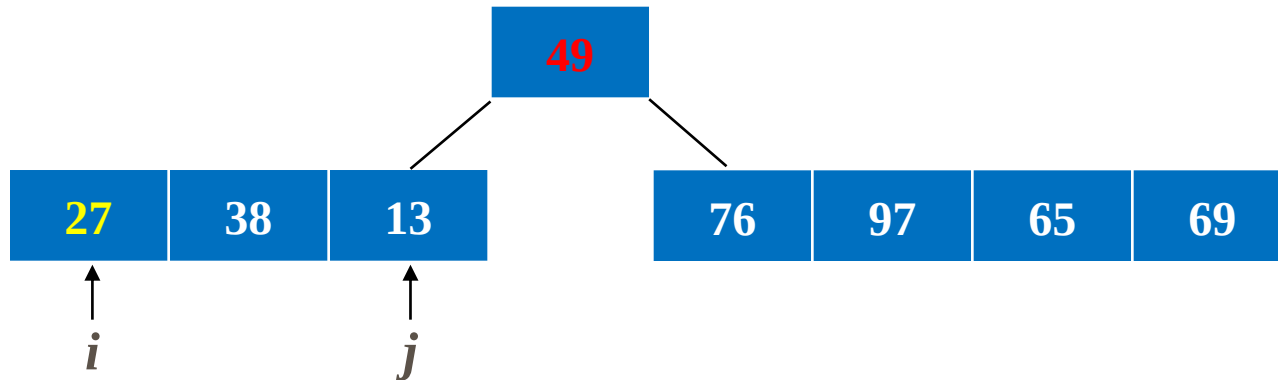
10.3.2 快速排序

```
void quickSort(int r[], int low, int high ) {  
    pos = partition(r, low, high );  
    quickSort(r, low, pos-1);  
    quickSort(r, pos+1,high );  
}
```



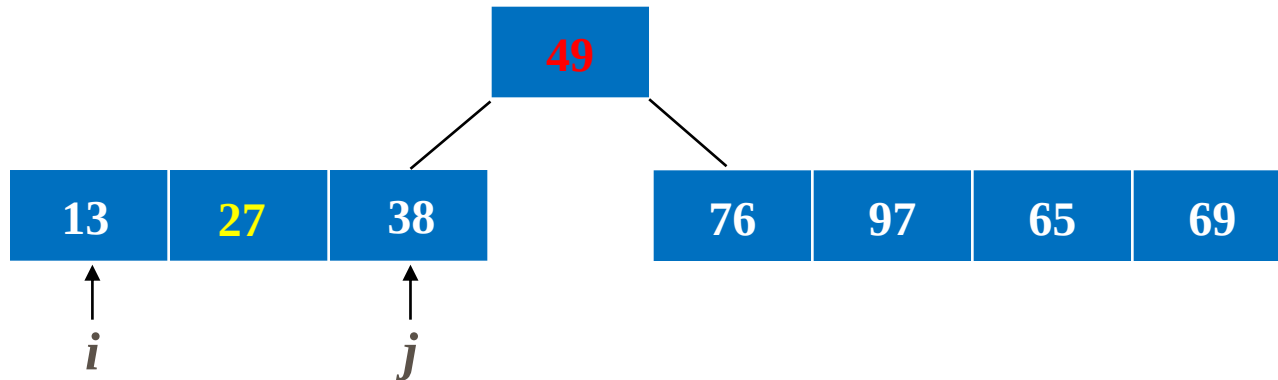
10.3.2 快速排序

```
void quickSort(int r[], int low, int high ) {  
    pos = partition(r, low, high );  
    quickSort(r, low, pos-1);  
    quickSort(r, pos+1,high );  
}
```



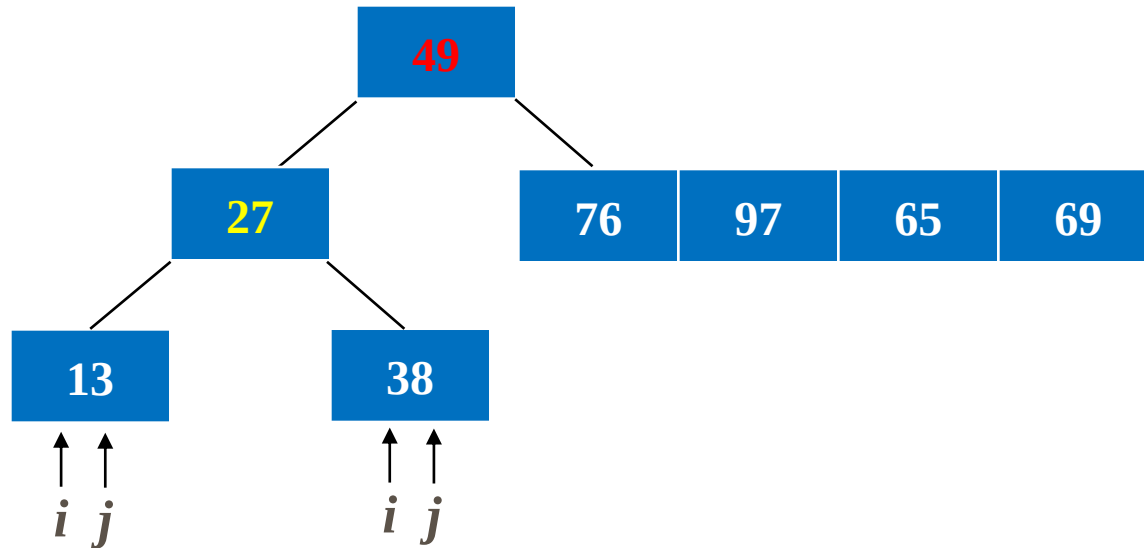
10.3.2 快速排序

```
void quickSort(int r[], int low, int high ) {  
    pos = partition(r, low, high );  
    quickSort(r, low, pos-1);  
    quickSort(r, pos+1,high );  
}
```



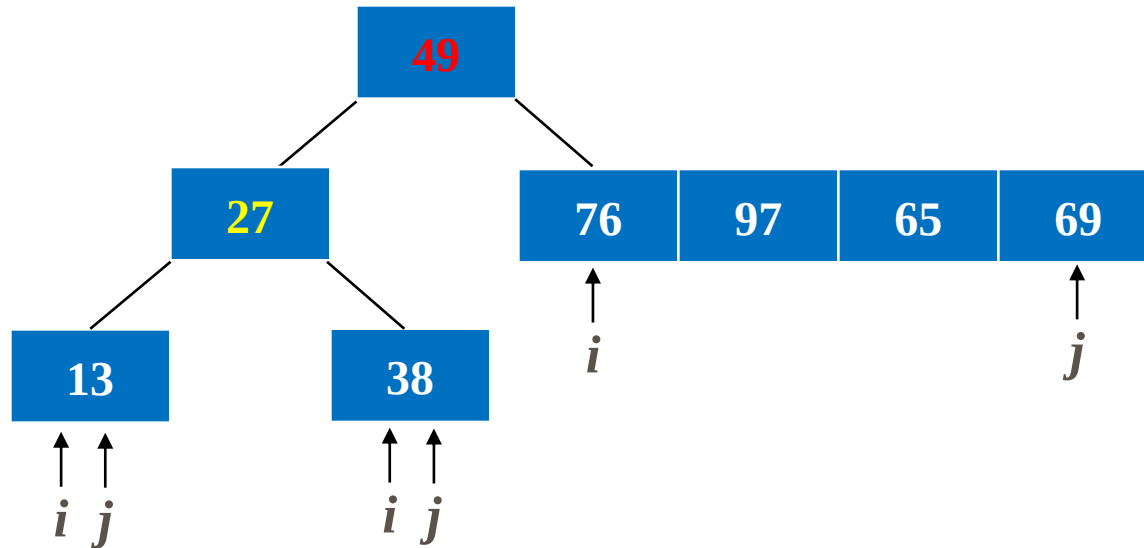
10.3.2 快速排序

```
void quickSort(int r[], int low, int high ) {  
    pos = partition(r, low, high );  
    quickSort(r, low, pos-1);  
    quickSort(r, pos+1,high );  
}
```



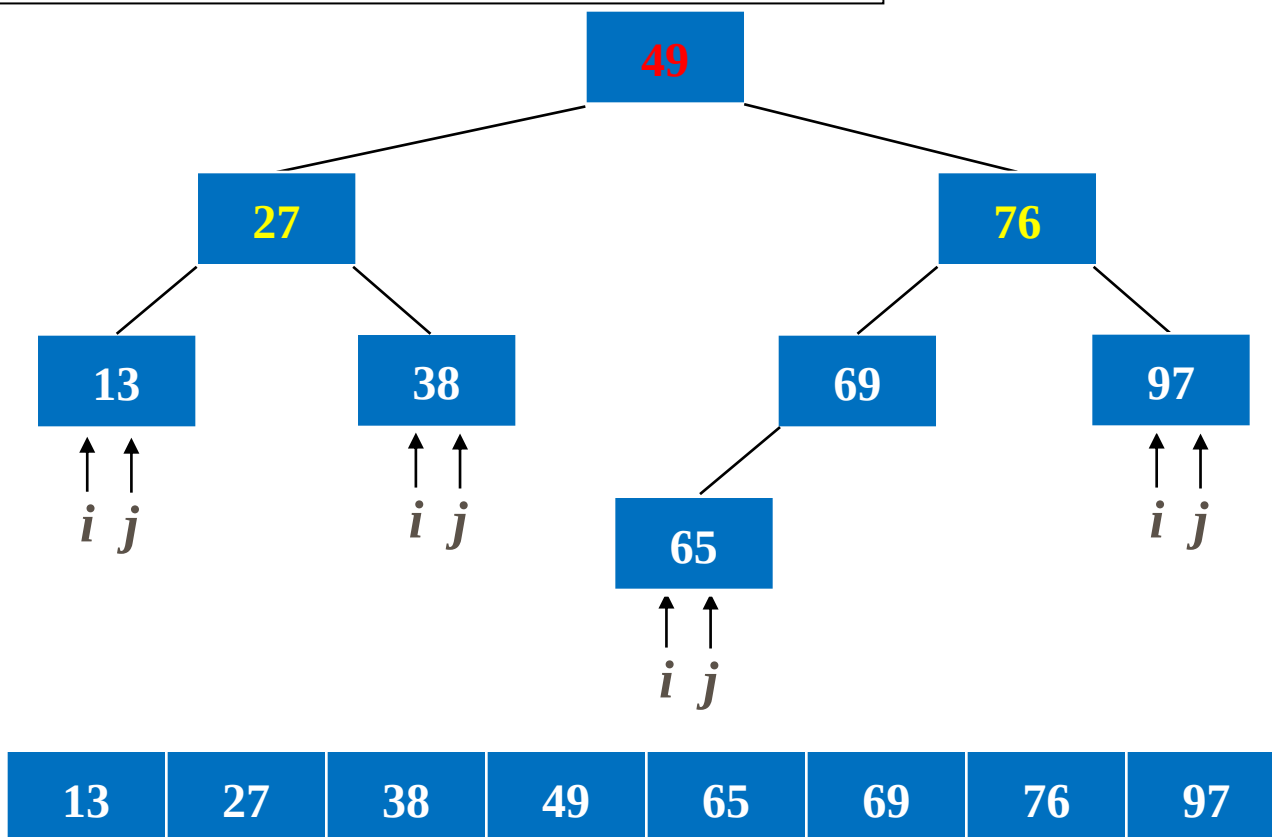
10.3.2 快速排序

```
void quickSort(int r[], int low, int high ) {  
    pos = partition(r, low, high );  
    quickSort(r, low, pos-1);  
    quickSort(r, pos+1,high );  
}
```



10.3.2 快速排序

```
void quickSort(int r[], int low, int high ) {  
    pos = partition(r, low, high );  
    quickSort(r, low, pos-1);  
    quickSort(r, pos+1,high );  
}
```



10.3.2 快速排序

关键问题(4):如何判别快速排序的结束?

解决方法:

若待排序列中只有一个记录, 显然已有序, 否则进行一次划分后, 再分别对分割所得的两个子序列进行快速排序(即递归处理)。

算法描述:

```
void quickSort(int r[], int low, int high ) {  
    if(low<high){ //长度大于1  
        pos = partition(r, low, high );  
        quickSort(r, low, pos-1);  
        quickSort(r, pos+1,high );  
    }  
}
```

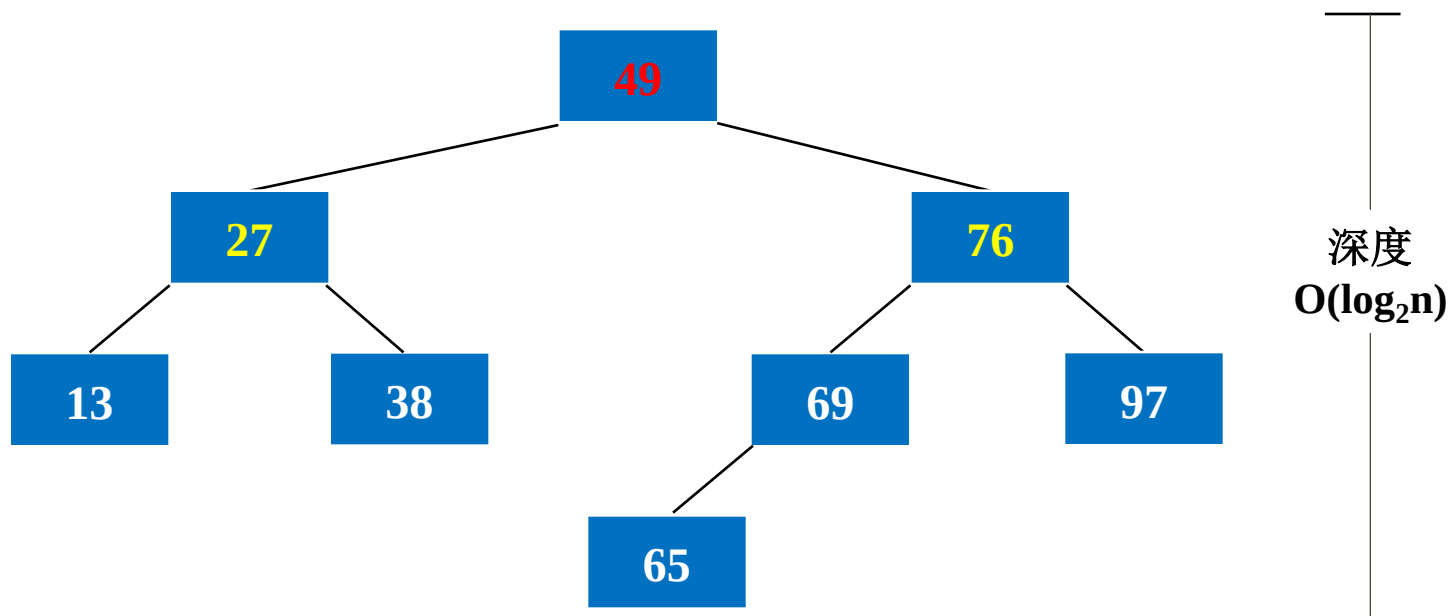
10.3.2 快速排序

优点： 因为每趟可以确定不止一个元素的位置，而且呈指数增加，所以特别快！

前提： 顺序存储结构

10.3.2 快速排序

时间复杂度



时间复杂度为: $O(n\log_2 n)$

10.3.2 快速排序

时间效率： $O(n\log_2 n)$ — 因为每趟确定的元素呈指数增加

空间效率： $O(\log_2 n)$ — 因为递归要用栈(存每层low, high和pivot)

稳定性：不稳定 — 因为有跳跃式交换。

10.3.2 快速排序

快速排序的不足和克服方法

最坏情况下时间复杂度为 $O(n^2)$ 级。如：在序列已是有序的情况下：10 20 30 40 50 60 70 80

[10] [20 30 40 50 60 70 80] 轴心 10

原因：支点选择不当

改进：随机选取支点或最左、最右、中间三个元素中的值处于中间的作为支点，通常可以避免最坏情况。

所以，快速排序在表已基本有序的情况下不合适。

在序列长度较短时，采用直接插入排序、冒泡排序等排序方法。

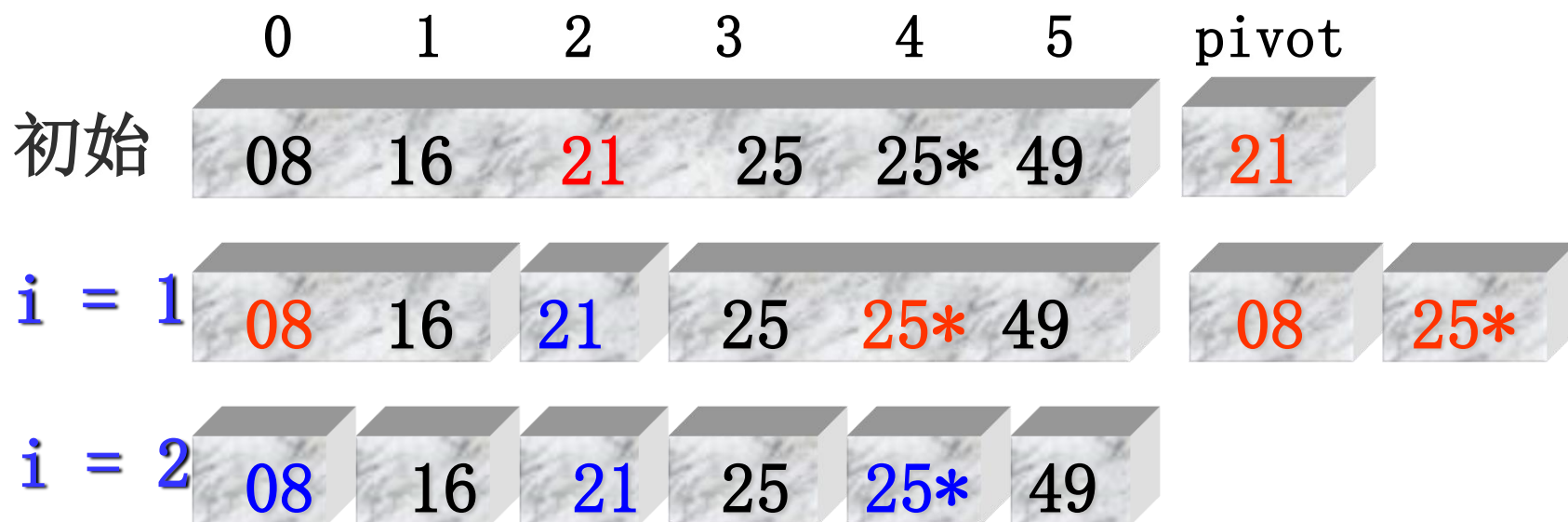
10.3.2 快速排序

取每个待排序对象序列的第一个对象、最后一个对象和位置接近正中的3个对象，取其关键字大小居中者作为基准对象。
(只要将该记录与L.r[low]互换, 算法10.6(b)不变。)

用第一个对象作为基准对象

	0	1	2	3	4	5	<i>pivot</i>
初始	08	16	21	25	25*	49	08
i=1	08	16	21	25	25*	49	16
i=2	08	16	21	25	25*	49	21
i=3	08	16	21	25	25*	49	25
i=4	08	16	21	25	25*	49	25*
i=5	08	16	21	25	25*	49	

10.3.2 快速排序



用居中关键字对象作为基准对象

10.4 选择排序

选择排序有多种具体实现算法：

1) 简单选择排序

2) 树形选择排序

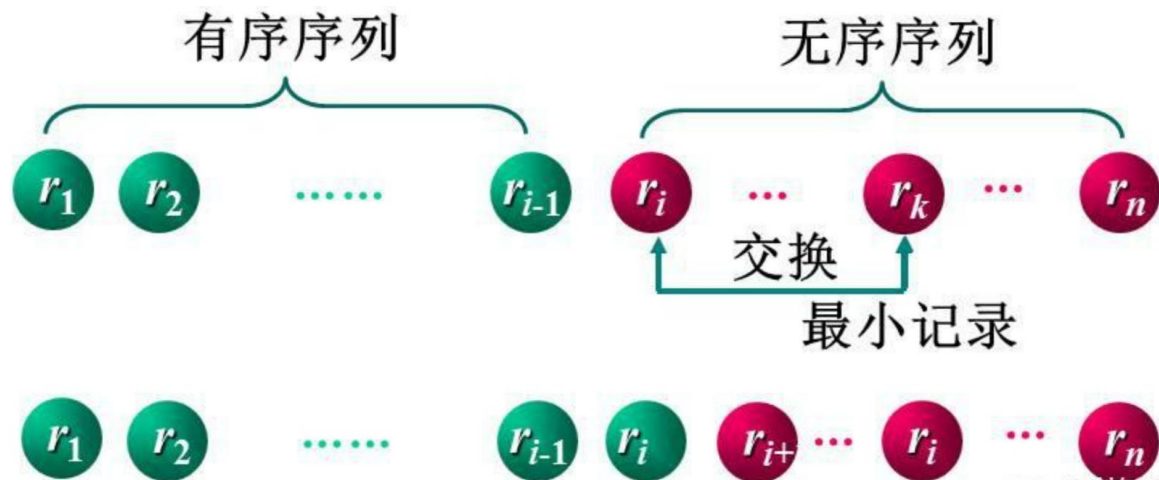
3) 堆排序 (重点)

10.4 选择排序

1、简单选择排序

算法思想：每经过一趟比较就找出一个最小值，与待排序列最前面的位置互换即可。

——首先，在 n 个记录中选择最小者放到 $r[1]$ 位置；然后，从剩余的 $n-1$ 个记录中选择最小者放到 $r[2]$ 位置；...如此进行下去，直到全部有序为止。



10.4 选择排序

1、简单选择排序

优点：实现简单

缺点：每趟只能确定一个元素，表长为 n 时需要 $n-1$ 趟

前提：顺序存储结构

10.4 选择排序

1、简单选择排序

例：关键字序列T= (21, 25, 49, 25*, 16, 08) ,

请给出简单选择排序的具体实现过程。

原始序列： 21, 25, 49, 25*, 16, 08

最小值 08 与
r[1] 交换位置

直接
选择
排序

第1趟 08, 25, 49, 25*, 16, 21

第2趟 08, 16, 49, 25*, 25, 21

第3趟 08, 16, 21, 25*, 25, 49

第4趟 08, 16, 21, 25*, 25, 49

第5趟 08, 16, 21, 25*, 25, 49

10.4 选择排序

1、简单选择排序

时间效率： $O(n^2)$ ——虽移动次数较少，但比较次数仍多。

空间效率： $O(1)$ ——没有附加单元（仅用到1个temp）

算法的稳定性：**不稳定**——因为排序时，25*到了25的前面。

10.4 选择排序

1、简单选择排序

```
Void SelectSort(SqList &L ) {  
    for (i=1; i<L.length; ++i){ //对顺序表L作简单选择排序  
        j = SelectMinKey(L,i); //在r[i...L.length]中选择最小记录并定位  
        if( i!=j ) r[i] ↔ r[j]; //与第i个记录交换  
    } //for  
} //SelectSort
```

10.4 选择排序

1、简单选择排序

选择排序的性能分析

移动次数：

最好情况（正序）：0次



10.4 选择排序

1、简单选择排序

选择排序的性能分析

移动次数：

最好情况（正序）：0次

最坏情况： $3(n-1)$ 次

比较次数：

$$\sum_{i=1}^{n-1} (n-i) = \frac{1}{2}n(n-1) = O(n^2)$$

简单选择排序的时间复杂度为 $O(n^2)$ 。

空间性能：需一个辅助空间。



10.4 选择排序

1、简单选择排序

讨论：能否利用（或记忆）首趟的 $n-1$ 次比较所得信息，从而尽量减少后续比较次数呢？

答：能！

树形选择排序和
堆排序

10.4 选择排序

2、树形选择排序

基本思想：与体育比赛时的淘汰赛类似。

首先对 n 个记录的关键字进行两两比较，得到 $\lceil n/2 \rceil$ 个优胜者(关键字小者)，作为第一步比较的结果保留下来。然后在这 $\lceil n/2 \rceil$ 个较小者之间再进行两两比较， $\dots$ ，如此重复，直到选出最小关键字的记录为止。

10.4 选择排序

2、树形选择排序

优点：减少比较次数，加快排序速度

缺点：空间效率低

例：关键字序列 $T = (21, 25, 49, 25^*, 16, 08, 63)$ ，

请给出树形选择排序的具体实现过程。

注：为便于自动处理，建议每个记录多开两个特殊分量：

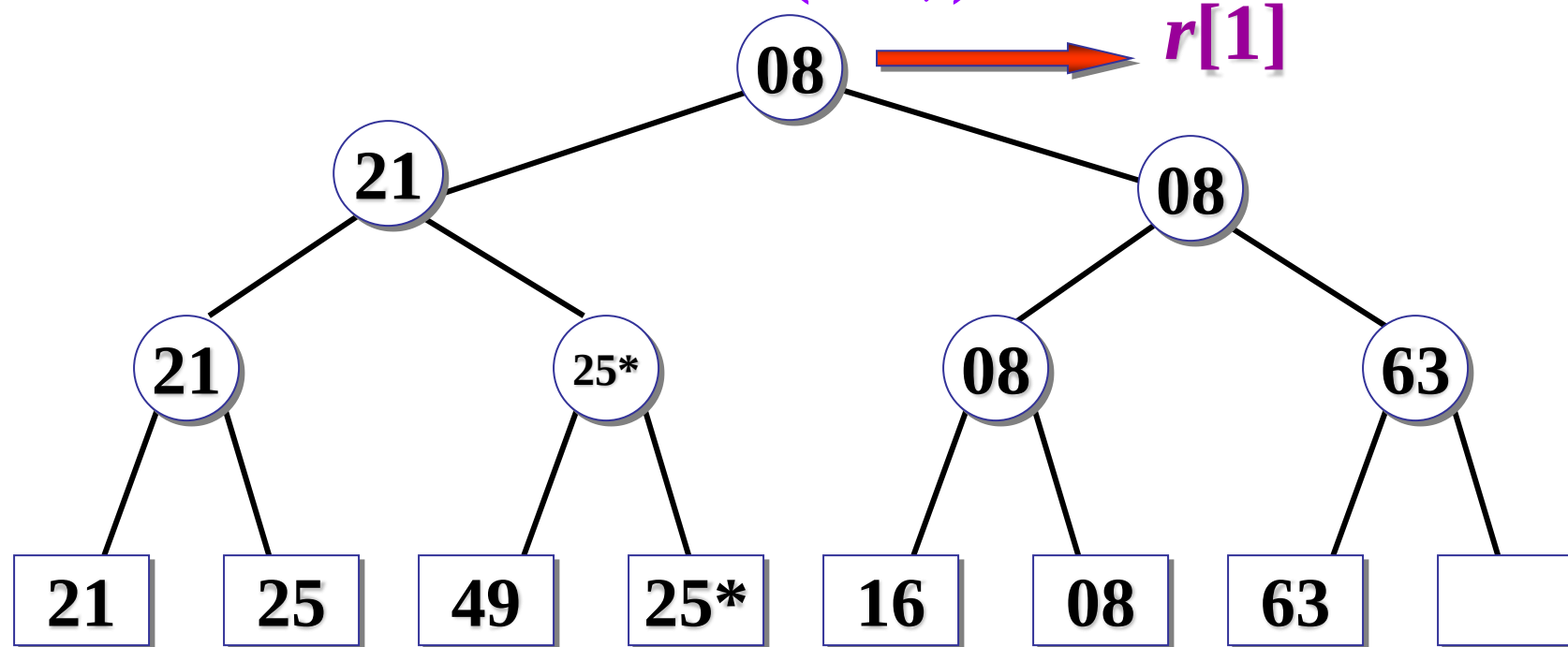
key	otherinfo	Index(结点位置编号)	Tag (是否参加比较)
-----	-----------	---------------	--------------

10.4 选择排序

2、树形选择排序

第一趟：

Winner (胜者)



初态：

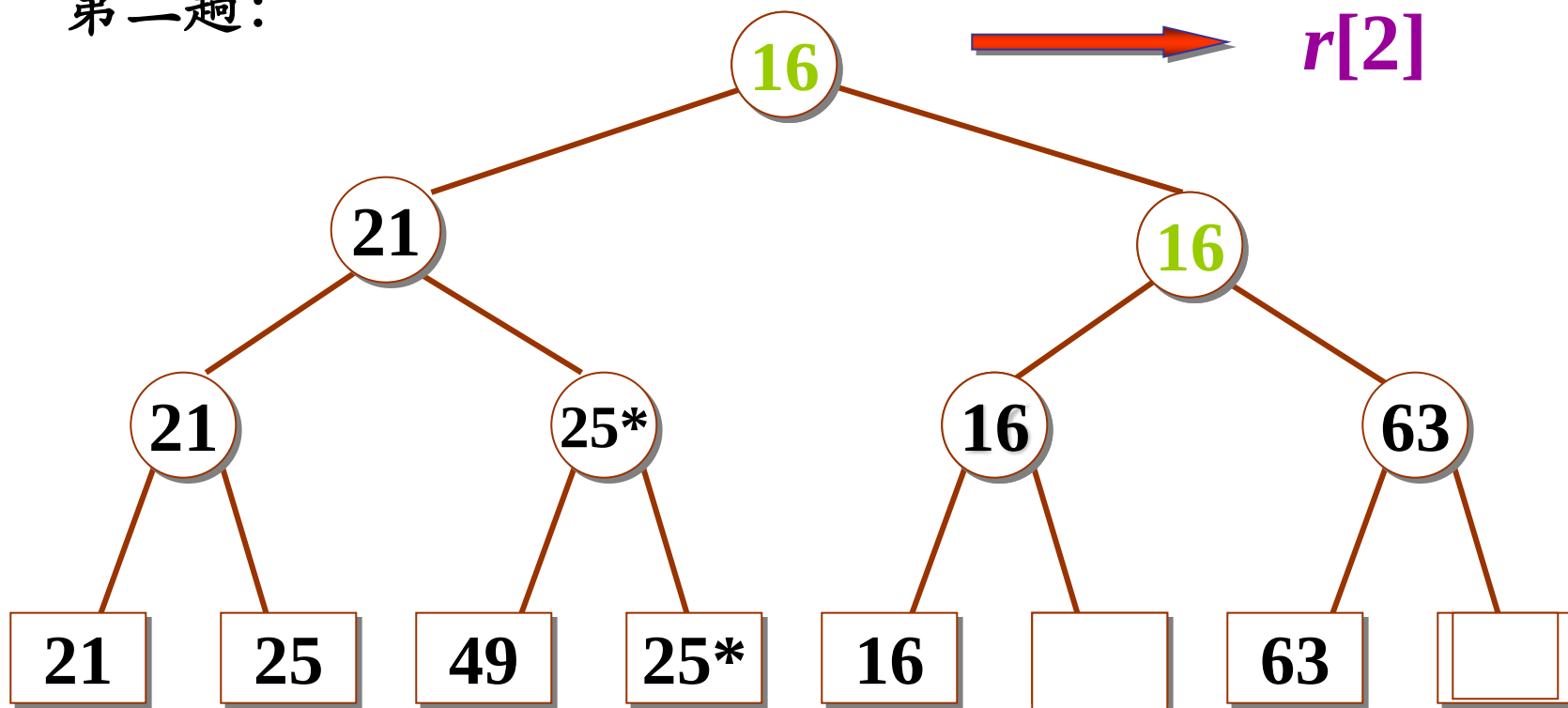
补足 2^k ($k = \lfloor \log_2 n \rfloor$)个叶子结点

10.4 选择排序

2、树形选择排序

第二趟：

Winner (胜者)



求次小值16时,只需比较2次,
即只比较 $\lceil \log_2 n \rceil - 1$ 次

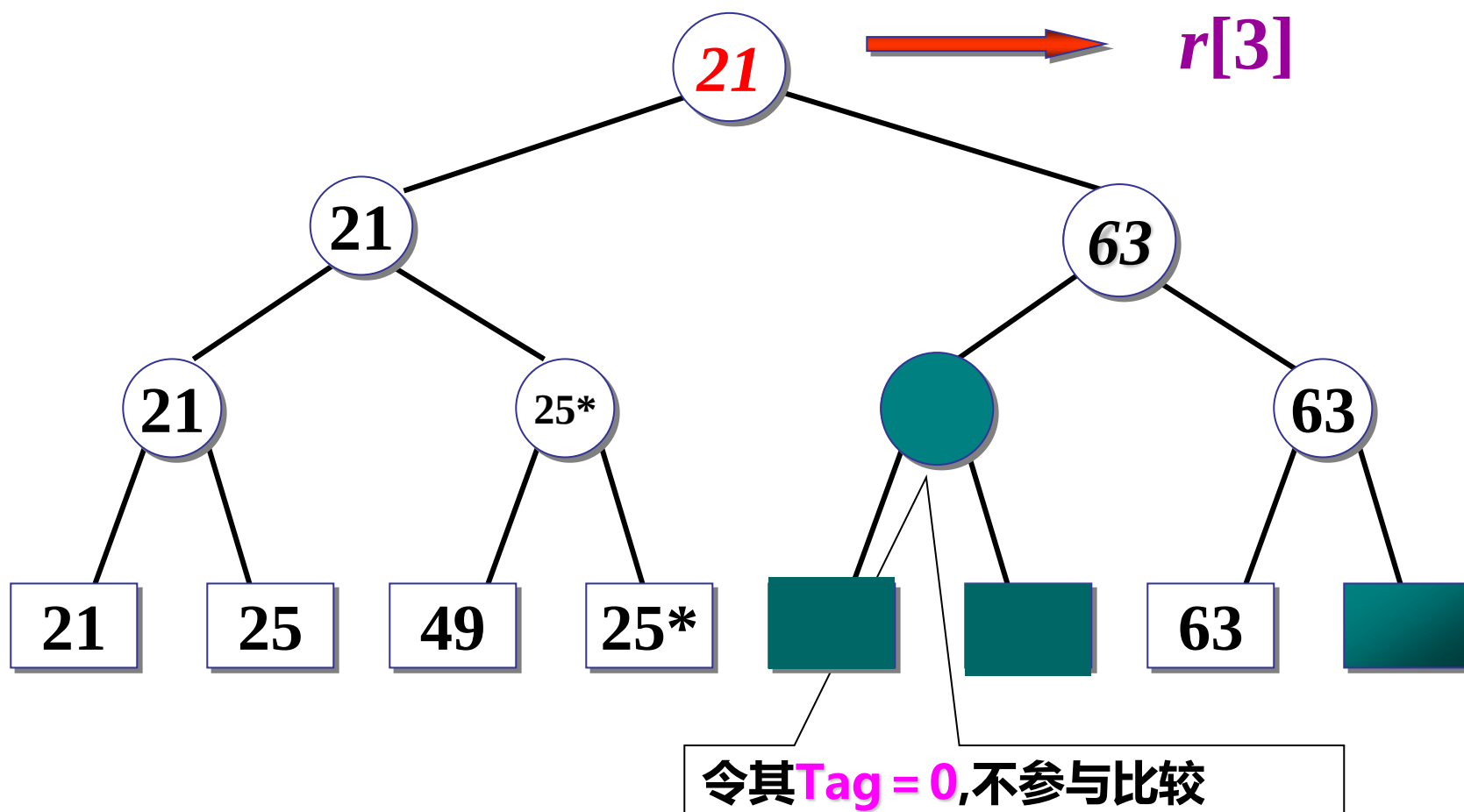
令其Tag=0,不参与
比较

10.4 选择排序

2、树形选择排序

第三趟：

Winner (胜者)

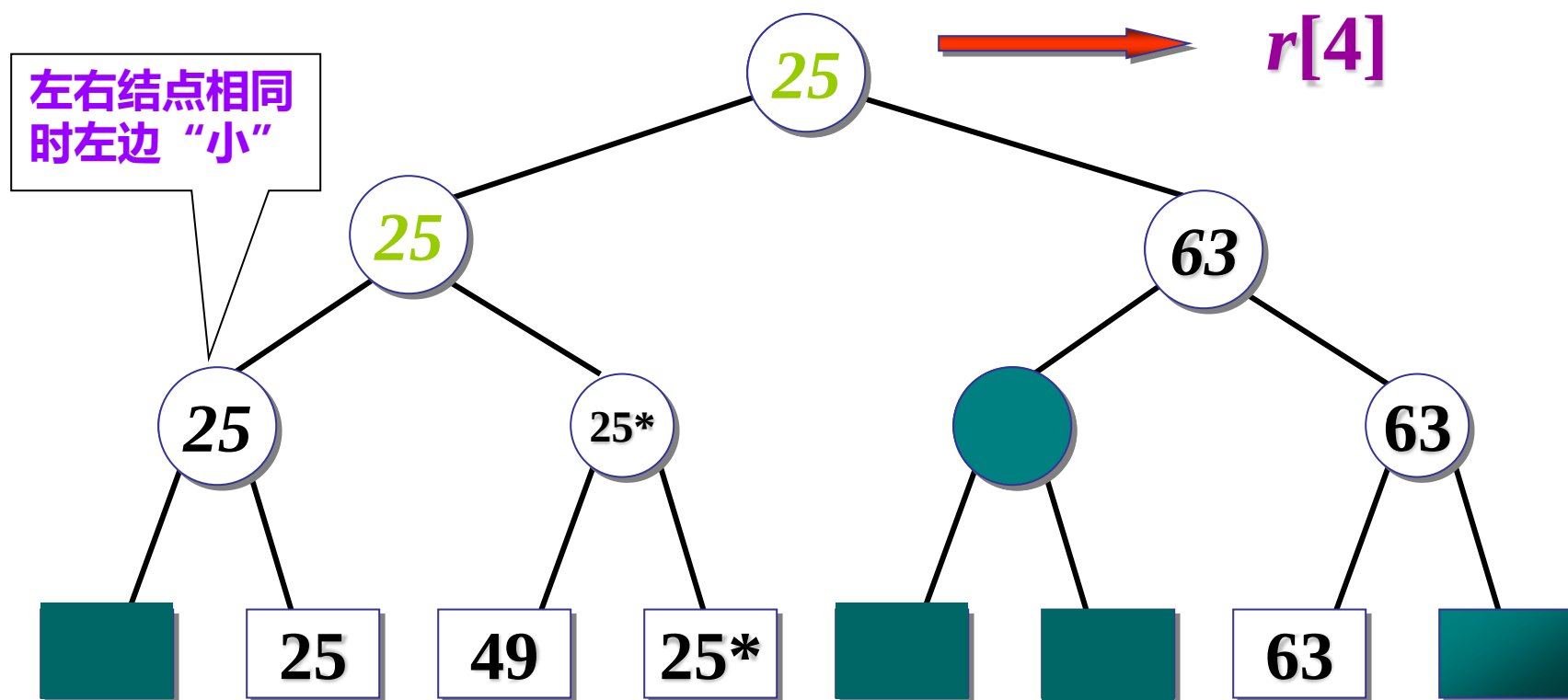


10.4 选择排序

2、树形选择排序

第四趟：

Winner (胜者)

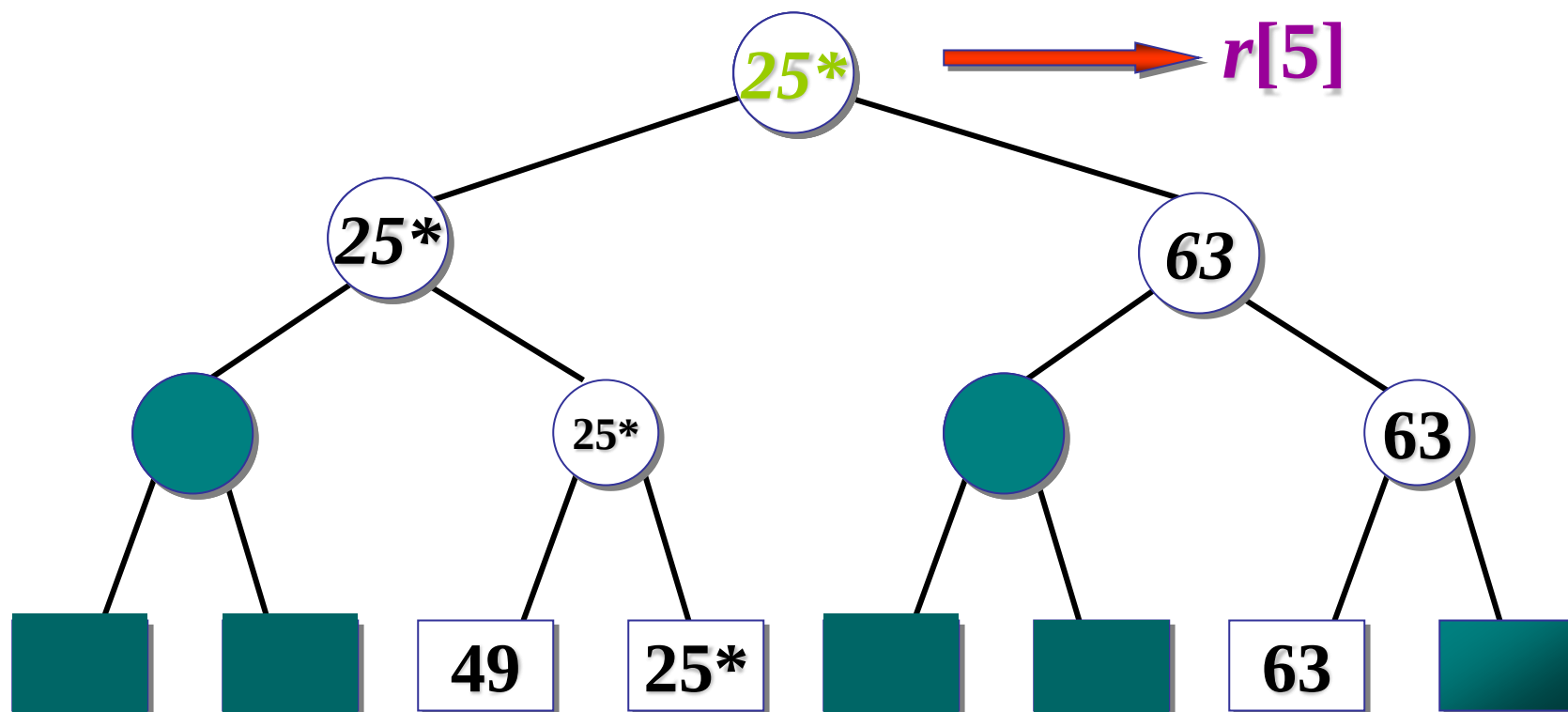


10.4 选择排序

2、树形选择排序

第五趟：

Winner (胜者)

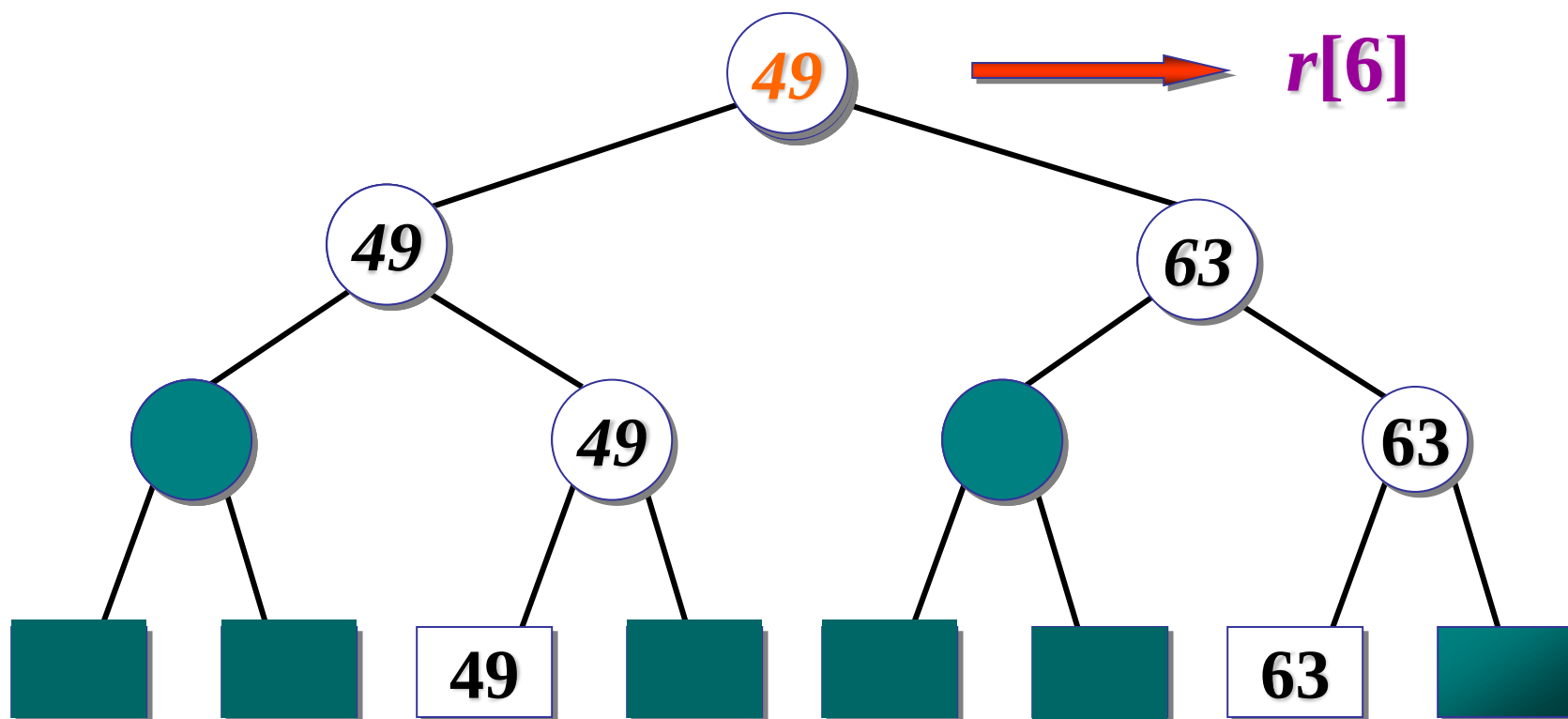


10.4 选择排序

2、树形选择排序

第六趟：

Winner (胜者)

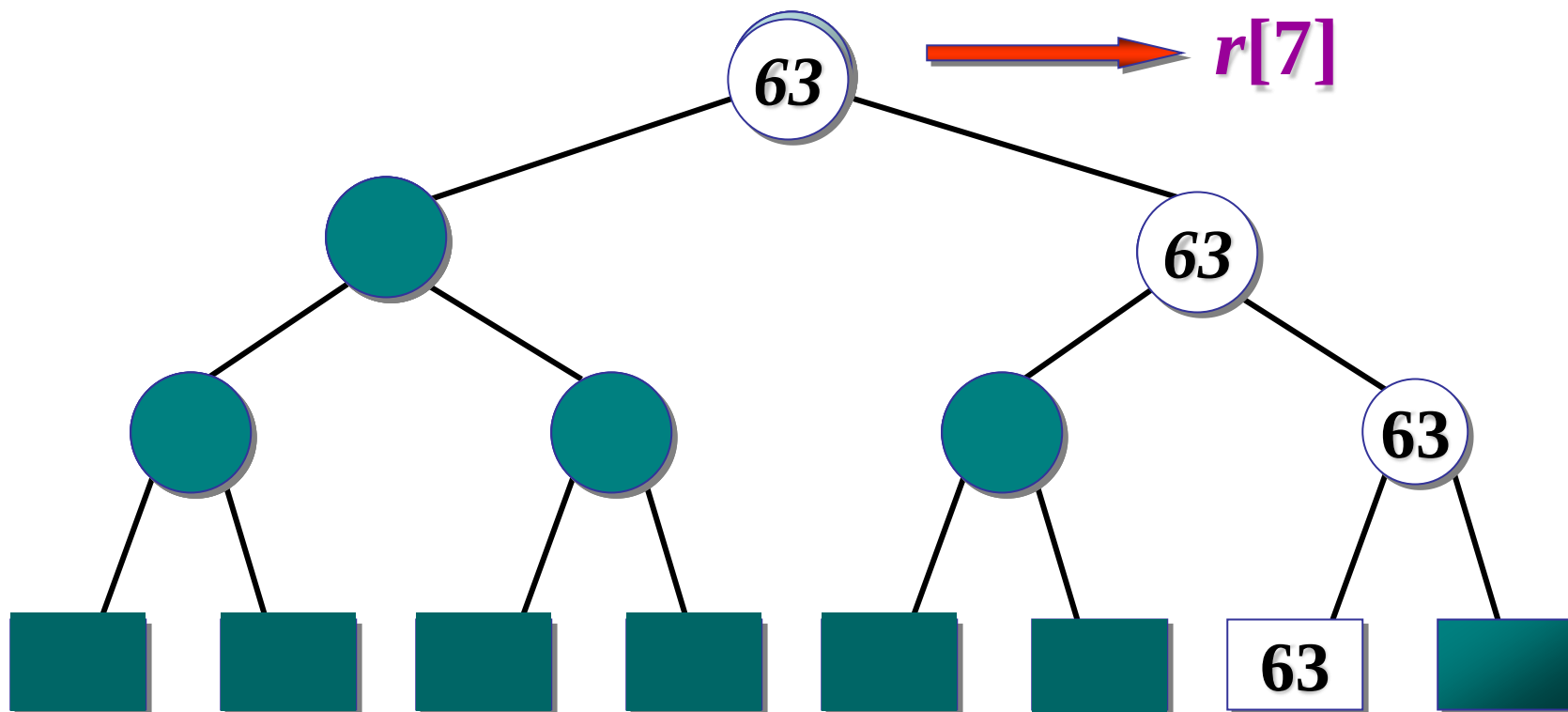


10.4 选择排序

2、树形选择排序

第七趟：

Winner (胜者)

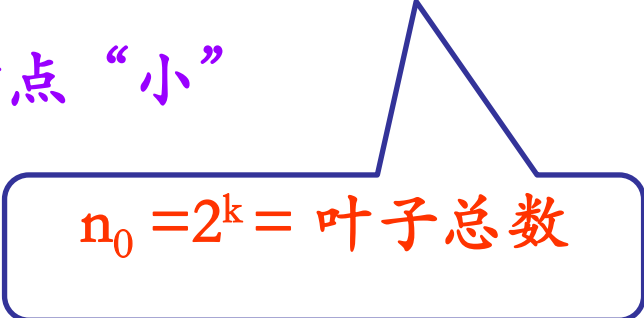


10.4 选择排序

2、树形选择排序

算法分析：

- 树形选择排序构成的树是**完全(满) 二叉树**，其深度为 $\lfloor \log_2 n \rfloor + 1$ ，其中 n 为待排序元素个数。
- 时间复杂度： $O(n \log_2 n)$ — n 个记录各自比较约 $\log_2 n$ 次
- 空间效率： $O(n)$ — 胜者树的附加内结点共有 $n_0 - 1$ 个！
- **稳定性**： 稳定 — 可事先约定左结点 “小”


$$n_0 = 2^k = \text{叶子总数}$$

10.4 选择排序

2、树形选择排序

讨论：树形选择排序巧妙利用了第一次和前若干次扫描的结果，加快了排序速度。还有没有同类方法？

答：有！

堆排序

第10章 内部排序

<https://www.cs.usfca.edu/galles/visualization/Algorithms.html>

David Galles Computer Science University of San Francisco	◦ AVL Trees (balanced binary search trees) ◦ Red-Black Trees ◦ Splay Trees ◦ Open Hash Tables (Closed Addressing) ◦ Closed Hash Tables (Open Addressing) ◦ Closed Hash Tables, using buckets ◦ Trie (Prefix Tree, 26-ary Tree) ◦ Radix Tree (Compact Trie) ◦ Ternary Search Tree (Trie with BST of children) ◦ B Trees ◦ B+ Trees • Sorting ◦ Comparison Sorting ▪ Bubble Sort ▪ Selection Sort ▪ Insertion Sort ▪ Shell Sort ▪ Merge Sort ▪ Quick Sort ◦ Bucket Sort ◦ Counting Sort ◦ Radix Sort ◦ Heap Sort • Heap-like Data Structures ◦ Heaps ◦ Binomial Queues ◦ Fibonacci Heaps ◦ Leftist Heaps ◦ Skew Heaps • Graph Algorithms ◦ Breadth-First Search ◦ Depth-First Search ◦ Connected Components ◦ Dijkstra's Shortest Path ◦ Prim's Minimum Cost Spanning Tree ◦ Topological Sort (Using Indegree array) ◦ Topological Sort (Using DFS) ◦ Floyd-Warshall (all pairs shortest paths) ◦ Kruskal Minimum Cost Spanning Tree Algorithm • Dynamic Programming ◦ Calculating nth Fibonacci number ◦ Making Change ◦ Longest Common Subsequence • Geometric Algorithms ◦ 2D Rotation and Scale Matrices ◦ 2D Rotation and Translation Matrices ◦ 2D Changing Coordinate Systems ◦ 3D Rotation and Scale Matrices ◦ 3D Changing Coordinate Systems • Others ... ◦ Disjoint Sets ◦ Huffman Coding (available in java version)
---	---

正在答疑
